

Regression analysis, resampling methods and gradient descent for the Runge function

Anne Kristin Furu

Department of Physics, University of Oslo

<https://github.com/Annekf/FYS-STK4155-Project1>

FYS-STK4155, fall 2026

Abstract

We study ordinary least squares (OLS), Ridge and Lasso regression on Runge’s function $f(x) = 1/(1+25x^2)$ on $[-1, 1]$, fitted with polynomials up to degree 15 on $n = 100$ noisy points. We assess the models with the bootstrap and with k -fold cross-validation, and we replace the closed-form solutions by gradient descent with momentum and the adaptive methods AdaGrad, RMSProp and Adam, both with the full gradient and with minibatches. Our own solvers reproduce the closed-form OLS and Ridge solutions, the analytical gradient matches automatic differentiation to machine precision ($\sim 10^{-15}$), and our proximal Lasso matches scikit-learn to $\sim 10^{-10}$. The bootstrap makes the bias–variance tradeoff explicit: the variance rises steeply with the polynomial degree while the bias stays low. Cross-validation selects the models cleanly, and OLS (degree 12, CV MSE ≈ 0.019) and Ridge (≈ 0.021) are essentially tied, with Lasso a little higher (≈ 0.027). On this smooth, low-dimensional problem the penalties buy little over a well-chosen OLS degree.

1 Introduction

Polynomial regression is the simplest setting in which the central tension of supervised learning appears: a more flexible model fits the training data better but generalises worse. Runge’s function

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1], \quad (1)$$

is a classic hard case for polynomial interpolation, with a sharp central peak that low-degree polynomials cannot follow and high-degree polynomials overfit. The general lesson is the bias–variance tradeoff, Section 2.10 of [7] and [6]. It is therefore a good testbed for regression and model selection.

In this project we (i) fit (1) with OLS and Ridge using our own closed-form code; (ii) study the bias–variance tradeoff with the bootstrap and select models with cross-validation; (iii) replace the closed form by gradient descent, with the gradient computed both analytically and by automatic differentiation, and extend it with momentum and the adaptive methods; (iv) solve Lasso, which has no closed form, by proximal gradient descent; and (v) compare OLS, Ridge and Lasso with cross-validation. Throughout we work with scaled data and a fixed random seed for reproducibility.

The report is organised as follows. Section 2 sets up the theory and methods, including the derivation of the bias–variance decomposition. Section 3 describes the code and the validation tests. Section 4 presents and discusses the results part by part, and Section 5 concludes.

2 Theory and methods

2.1 Ordinary least squares

We model the data as $\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ with independent noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ and a design matrix $\mathbf{X}$ whose columns are powers of x . OLS minimises the mean squared error

$$C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2, \quad (2)$$

with the closed-form minimiser $\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Assuming the model is correct, the estimator is unbiased and has variance

$$\mathbb{E}[\hat{\boldsymbol{\theta}}] = \boldsymbol{\theta}, \quad \text{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}, \quad (3)$$

so the diagonal of (3) gives a confidence interval for each coefficient, with σ^2 estimated from the residuals as $\hat{\sigma}^2 = \|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}\|_2^2 / (n - p)$. We solve the normal equations through the singular value decomposition (SVD) rather than an explicit inverse, for numerical stability. The quality of the fit is measured by the MSE and by the R^2 score. Features are standardised on the training statistics and the target is centred, so no test information leaks into the fit.

2.2 Ridge regression and singular-value shrinkage

Ridge adds an ℓ_2 penalty, $C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \|\boldsymbol{\theta}\|^2$, with the closed form

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}. \quad (4)$$

The factor n comes from the $1/n$ in the data term; scikit-learn omits it, so its `alpha` equals $n\lambda$. Writing $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$, the Ridge coefficient in singular direction i is the OLS one multiplied by the filter factor

$$f_i = \frac{s_i^2}{s_i^2 + n\lambda}, \quad (5)$$

so directions with large singular values are kept and small-singular-value directions are shrunk toward zero (Section 3.8 of [7]). Ridge regression goes back to [8]. These small-singular-value directions are exactly the ill-conditioned ones that make the OLS coefficients unstable.

2.3 Lasso

Lasso [11] replaces the penalty by the ℓ_1 norm, $\lambda \|\boldsymbol{\theta}\|_1$, which has no closed form and is not differentiable at $\theta_j = 0$; its subdifferential there is $[-1, 1]$. We solve it by proximal gradient descent (ISTA): a gradient step on the smooth data term followed by the soft thresholding operator $S_t(z) = \text{sign}(z) \max(|z| - t, 0)$, the proximal operator of the ℓ_1 penalty, which drives coefficients exactly to zero and so yields a sparse model. Algorithm 1 states the solver; it is implemented in `LASSO.py`.

Algorithm 1 Proximal gradient descent (ISTA) for Lasso

- 1: $\boldsymbol{\theta} \leftarrow \mathbf{0}$; step $\gamma = 1/L$ with $L = \lambda_{\max}(\frac{2}{n} \mathbf{X}^T \mathbf{X})$
 - 2: **repeat**
 - 3: $\mathbf{g} \leftarrow \frac{2}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$ ▷ gradient of the smooth term
 - 4: $\boldsymbol{\theta} \leftarrow S_{\gamma\lambda}(\boldsymbol{\theta} - \gamma\mathbf{g})$ ▷ soft-threshold, the ℓ_1 prox
 - 5: **until** $\|\Delta\boldsymbol{\theta}\| < \text{tol}$
-

2.4 The bias–variance decomposition

Assume the data are generated by $\mathbf{y} = f(\mathbf{x}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, and let $\tilde{\mathbf{y}}$ be the prediction of a model fitted on a random training set. Adding and subtracting $\mathbb{E}[\tilde{\mathbf{y}}]$ inside the expected squared error and grouping the terms as $\mathbf{y} - \tilde{\mathbf{y}} = (f - \mathbb{E}[\tilde{\mathbf{y}}]) + \varepsilon + (\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}})$, the three squares survive and the three cross terms vanish, each by one assumption: zero-mean noise, f non-stochastic, and the independence of the test-point noise from the training set. The result is

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \underbrace{\frac{1}{n} \sum_i (f_i - \mathbb{E}[\tilde{y}_i])^2}_{\text{Bias}^2} + \underbrace{\frac{1}{n} \sum_i \text{Var}(\tilde{y}_i)}_{\text{Variance}} + \sigma^2. \quad (6)$$

The expectation runs over both the noise and the random training set. The bias measures how far the average model is from the truth, the variance how much the model moves between training sets, and σ^2 is the irreducible noise. When the true f is unknown we estimate the bias against the noisy test targets, so the measured bias absorbs σ^2 ; the printed identity error = bias + variance then holds per point, with the bias an upper estimate.

2.5 Resampling

The bootstrap estimates (6) by refitting the model on resamples of the training set and reading off the mean, the spread and the error of the predictions on a fixed test set. Cross-validation instead splits the data into k folds, using each fold once as the test set, and averages the test MSE. Any scaling is fitted inside each fold, on the training folds only, so the held-out fold stays unseen.

2.6 Gradient descent and its extensions

Gradient descent updates $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma \nabla C(\boldsymbol{\theta}_k)$ with the analytical gradient $\nabla C_{\text{OLS}} = \frac{2}{n} \mathbf{X}^T (\mathbf{X} \boldsymbol{\theta} - \mathbf{y})$. It converges only for $\gamma < 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of the Hessian $\frac{2}{n} \mathbf{X}^T \mathbf{X}$, is fastest at $\gamma^* = 2/(\lambda_{\max} + \lambda_{\min})$, and slows as the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ grows. Momentum, AdaGrad, RMSProp and Adam [3, 9] accelerate or stabilise this. The adaptive methods give each coordinate its own step $\gamma/\sqrt{v_j}$, so γ becomes a length in parameter space and cannot violate the stability bound. Stochastic gradient descent replaces the full gradient by a minibatch gradient inside an epoch loop [4]; the minibatch gradient is unbiased with noise $\propto 1/\sqrt{M}$, so at a constant learning rate the iterate equilibrates in a cloud whose radius scales with γ , and a decaying schedule $\gamma_t = t_0/(t + t_1)$ that satisfies the Robbins–Monro conditions is needed to converge. Algorithm 2 states the minibatch loop, implemented in `optimisers.py`; the per-step update `optimiser_step` is any of plain, momentum, AdaGrad, RMSProp or Adam.

Algorithm 2 Minibatch stochastic gradient descent

```

1:  $\boldsymbol{\theta} \leftarrow \mathbf{0}$ ; choose batch size  $M$ , epochs  $E$ , schedule  $(t_0, t_1)$ ;  $t \leftarrow 0$ 
2: for epoch = 1 to  $E$  do
3:   shuffle the  $n$  samples into batches of size  $M$ 
4:   for each batch  $B$  do
5:      $t \leftarrow t + 1$ ;  $\gamma_t \leftarrow t_0/(t + t_1)$ 
6:      $\mathbf{g} \leftarrow \frac{2}{|B|} \mathbf{X}_B^T (\mathbf{X}_B \boldsymbol{\theta} - \mathbf{y}_B)$  ▷ gradient on the minibatch
7:      $\boldsymbol{\theta} \leftarrow \text{optimiser\_step}(\boldsymbol{\theta}, \mathbf{g}, \gamma_t)$ 
8:   end for
9: end for
```

2.7 Automatic differentiation

Automatic differentiation applies the exact chain rule through the code; it is neither symbolic nor a finite difference, so it carries no truncation error, and in reverse mode a full gradient costs only two

to three cost evaluations [1, 2]. We use it to check our analytical gradient.

3 Code structure and validation

The code is organised in three folders: **code** (reusable modules and the notebooks), **results** (figures) and **report**. The modules separate the data generation, the three regression methods, the optimisers and the resampling tools, and the five notebooks import them, one per pair of project parts. The design keeps the optimiser, the data loop and the gradient independent: the optimiser knows nothing about the data, the data loop nothing about the optimiser, and the gradient is one function handed either all rows or a minibatch, so the same optimiser serves the full-gradient and the stochastic runs unchanged. The numerical stack is NumPy [5], scikit-learn [10] and JAX [2]. The reusable modules are adapted from our own weekly-exercise solutions and from the course lecture notebooks [7]. Table 1 records, for each module, the weekly exercise it is adapted from and the project parts it serves; every module repeats this source in a comment at the top of the file. All randomness uses the seed 2026.

Table 1: Provenance of the reusable code modules: the weekly exercise each is adapted from, and the project parts it serves. The Lasso solver is our own implementation, verified against scikit-learn.

Module	Adapted from	Parts
<code>Get_data.py</code>	week-38 data and standardisation helpers	a–i
<code>OLS.py</code>	week-35 SVD / pseudoinverse OLS	a
<code>Ridge.py</code>	week-37–38 closed-form Ridge	b
<code>Resample.py</code>	week-36 bootstrap and own k -fold	c, d
<code>optimisers.py</code>	week-37–38 optimiser and minibatch loops	e, f, h
<code>helper.py</code>	week-37–38 gradient and Hessian helpers	e
<code>LASSO.py</code>	own ISTA / proximal gradient descent	g

Following the recommendation that a code be checked against known results, we validate every component: our SVD and pseudoinverse OLS agree with each other and with scikit-learn [10]; the analytical gradient matches `jax.grad` to about 10^{-15} for both OLS and Ridge; the Ridge closed form matches scikit-learn once `alpha = nλ`; our proximal Lasso matches scikit-learn to about 10^{-10} ; and our own k -fold reproduces the scikit-learn cross-validation. The closed-form OLS and Ridge solutions are the tests of the gradient-descent code. Each notebook is run top to bottom from a fresh kernel, and the seed 2026 is stated so that every figure can be regenerated.

4 Results and discussion

4.1 OLS (part a)

Figure 2 shows the training and test MSE against the polynomial degree. The training MSE falls monotonically toward the noise floor $\sigma^2 = 0.01$, while the test MSE has a broad minimum around degrees 6–12 and rises again at high degree as the model fits the noise. The fitted parameters grow and oscillate as the degree rises (Figure 4): the largest coefficient is a few tenths at degree 5 but reaches several hundred at degree 15, the signature of variance. Scaling does not change the OLS predictions, but it controls the conditioning: Figure 3 shows that the condition number of $\mathbf{X}^T \mathbf{X}$ still reaches $\sim 10^{10}$ at degree 15 even after standardisation, because standardisation removes the difference in column magnitudes but not the strong correlation between the polynomial columns. More data pushes the overfitting to higher degree and lowers the test error. Repeating the degree sweep at $\sigma = 0.05, 0.1, 0.2$ (Figure 1) shows the conclusions are robust to the noise level: more noise raises the error floor and pulls the useful degree slightly lower, but the U-shape and the bias–variance reading are unchanged.

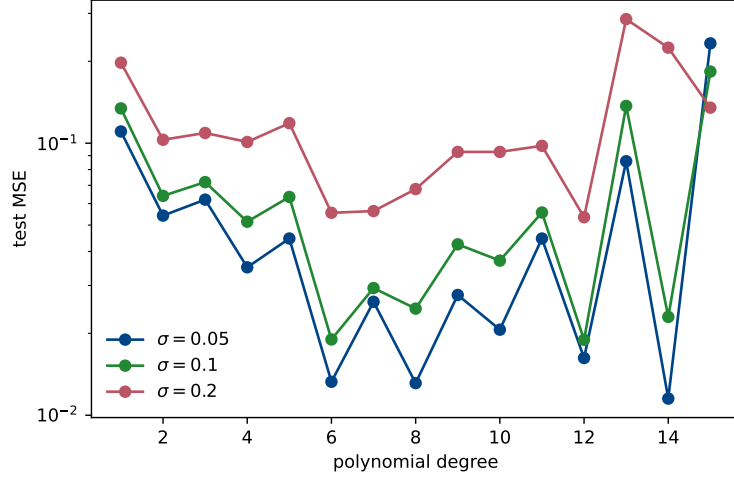


Figure 1: Test MSE against polynomial degree at three noise levels. More noise raises the error floor and pulls the useful degree slightly lower, while the U-shape and the degree window stay the same ($n = 100$, seed 2026).

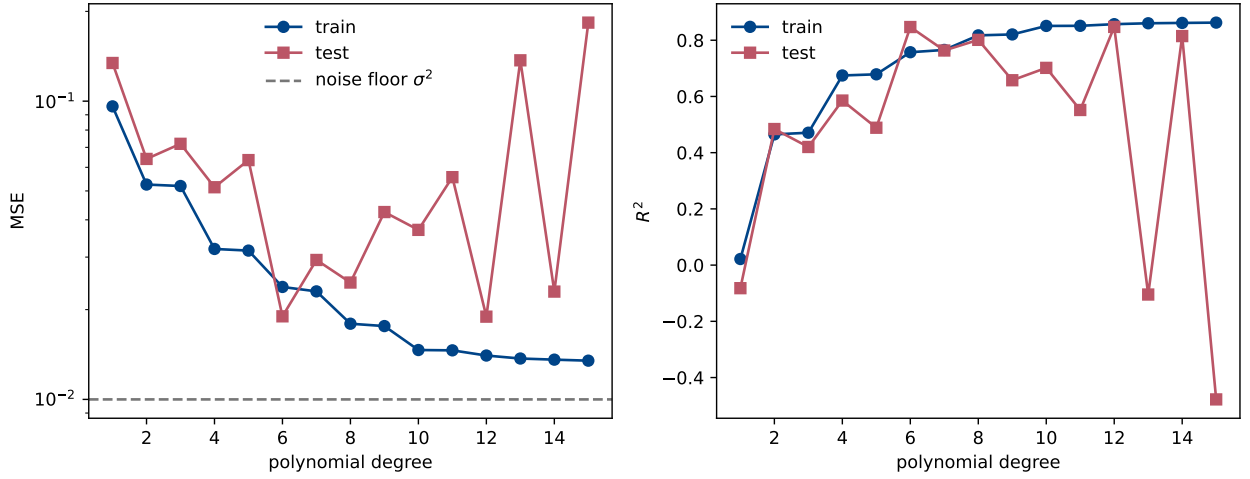


Figure 2: The training MSE falls monotonically to the noise floor σ^2 , while the test MSE is minimal in a broad region around degrees 6–12 and rises again as the variance takes over. Left: MSE (log scale); right: R^2 .

4.2 Ridge (part b)

A small penalty lowers the test error below OLS at high degree, while a large penalty underfits. Figure 5 illustrates the mechanism through the filter factors (5): directions with a large singular value are untouched, and small-singular-value directions are shrunk to zero. Ridge does not change the model; it changes which directions the fit trusts, damping the noisy directions that made the OLS coefficients blow up.

4.3 Bias–variance with the bootstrap (part c)

Figure 6 decomposes the bootstrap test error into squared bias and variance. The bias is low and roughly flat, the variance rises steeply with the degree, and their sum gives the U-shaped test error. Repeating for $n = 40, 100, 400$ shows that more data lowers the variance and shifts the overfitting to higher degree, while the bias barely moves: data reduces variance, not bias. This reproduces the textbook bias–variance account (Chapter 7 of [6], Section 2.11 of [7]): at high model complexity it is the variance, not the bias, that drives the test error, and the growth of the OLS coefficients with

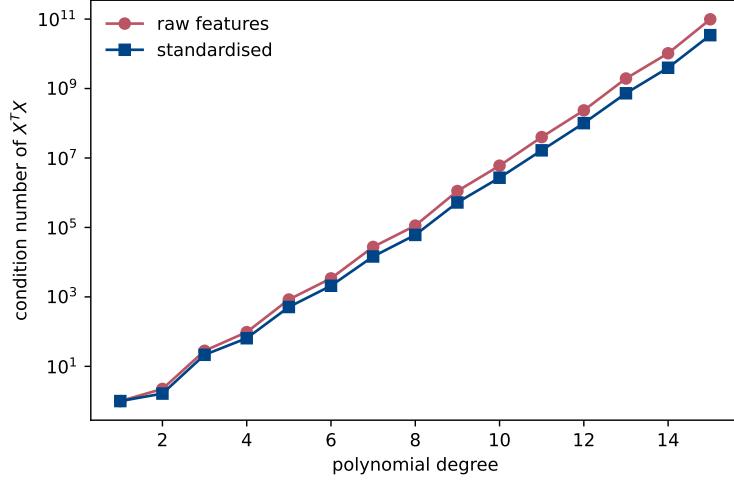


Figure 3: Even after standardisation the condition number of $\mathbf{X}^T \mathbf{X}$ reaches $\sim 10^{10}$ at degree 15: scaling removes the difference in column magnitudes, not the correlation between the polynomial columns.

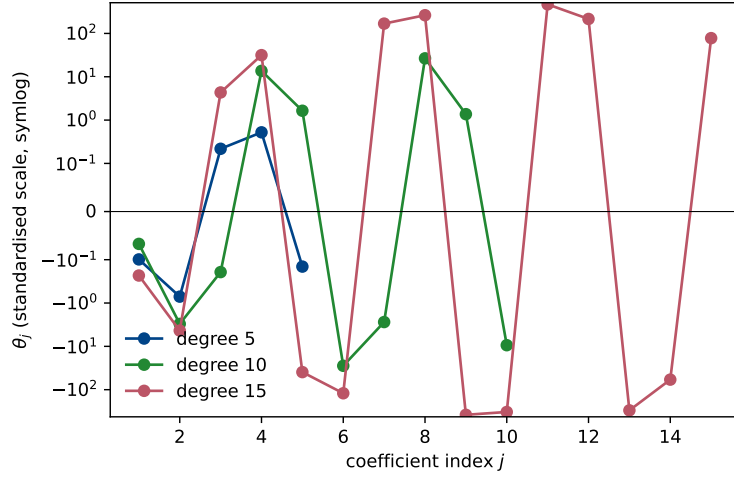


Figure 4: The fitted OLS coefficients θ_j (standardised scale, symlog axis) at degrees 5, 10 and 15. Low degrees stay near zero; at degree 15 the coefficients grow to several hundred and alternate in sign, the hallmark of overfitting.

degree is the Runge phenomenon in regression form.

4.4 Cross-validation (part d)

Our own k -fold and the scikit-learn cross-validation agree closely, and both track the bootstrap error and point to the same degree window (Figure 7). At high degree the bootstrap error explodes while the cross-validation stays flat, because a bootstrap resample can be nearly rank-deficient, which destabilises OLS, whereas k -fold keeps distinct points. The selected Ridge penalty is stable between $k = 5$ and $k = 10$.

4.5 Gradient descent and the adaptive methods (parts e, f)

The analytical and automatic gradients agree to machine precision, so either can drive the descent. Figure 8 confirms the stability edge: just below $2/\lambda_{\max}$ the iteration converges, just above it diverges, and γ^* is fastest. On the harder degree-6 problem ($\kappa \approx 2000$), momentum and Adam reach the tolerance in about 1600 and 1300 iterations against 17000 for plain gradient descent (Figure 9);

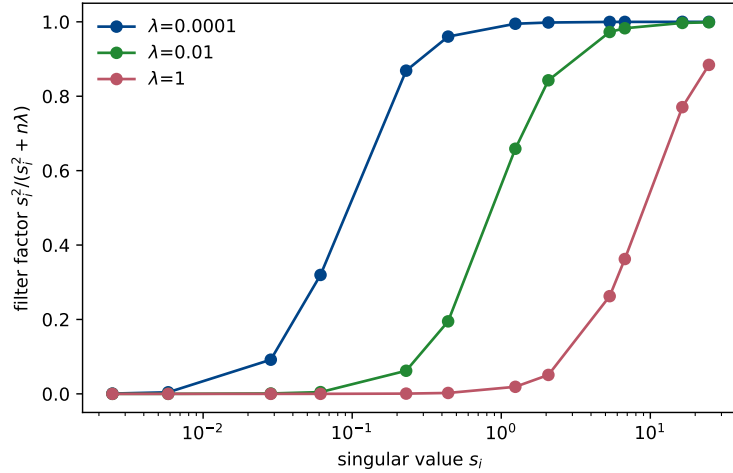


Figure 5: Ridge keeps the large-singular-value directions and shrinks the small ones toward zero through the filter factor $s_i^2/(s_i^2 + n\lambda)$; a larger λ shrinks more of them.

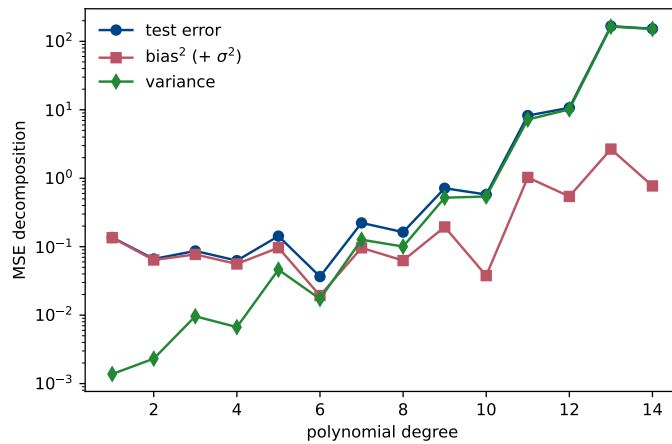


Figure 6: The variance rises steeply with the degree while the bias stays low, so the bootstrap test error is a U with a flat minimum ($n = 100$, 100 resamples, seed 2026).

AdaGrad and RMSProp do not reach it at a constant rate. Adam converges over the widest range of learning rates and is the least sensitive to the choice. A fair comparison of the optimisers must hold the rest fixed: the same data and starting point, the learning rate tuned per method since each has its own best γ , the same stopping tolerance, and a budget measured in gradient evaluations rather than raw iterations.

4.6 Lasso (part g)

JAX returns 1 for the derivative of $|\theta|$ at zero, a valid subgradient since $1 \in [-1, 1]$; proximal gradient descent is the cleaner tool and sets coefficients exactly to zero. Figure 10 shows the key difference between the penalties: Lasso removes coefficients one by one as λ grows, ending with a sparse model, while Ridge keeps all of them and only shrinks. In exact arithmetic ISTA sends the dropped coefficients to zero; with a finite iteration count it leaves them near zero (for example 2×10^{-4}), so we count a coefficient as zero below a threshold of 10^{-8} , whereas scikit-learn’s coordinate descent returns exact zeros. The difference is the solver’s stopping rule, not the model.

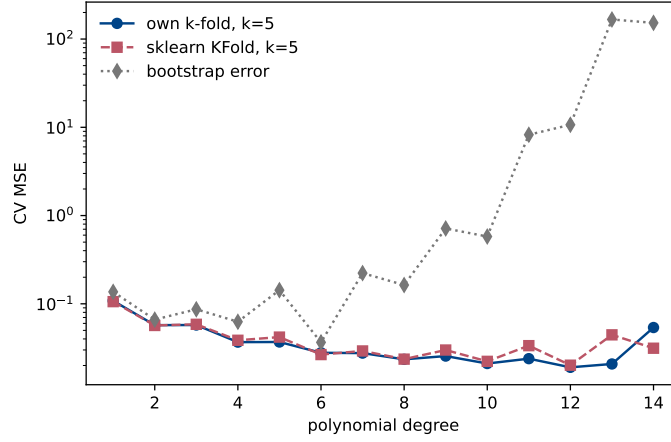


Figure 7: Own k -fold and scikit-learn agree and stay flat, while the bootstrap error explodes at high degree, where a resample becomes nearly rank-deficient and destabilises OLS.

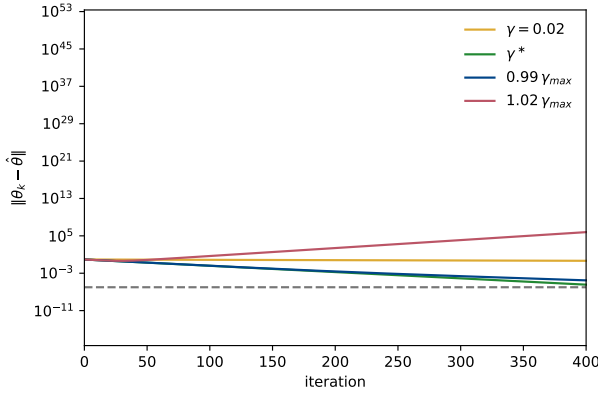


Figure 8: Plain gradient descent converges just below $2/\lambda_{\max}$ and diverges just above it; γ^* is fastest (degree 4).

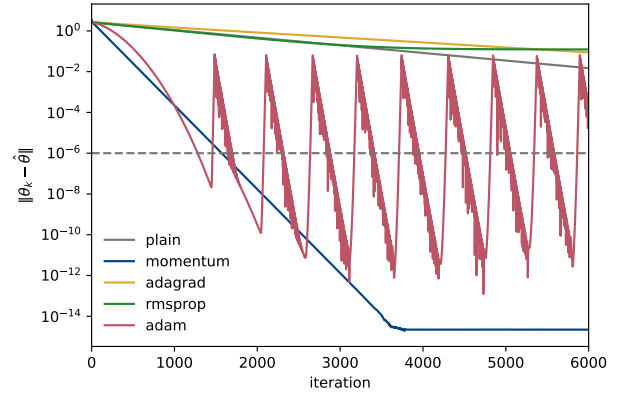


Figure 9: Momentum and Adam reach the closed form in about ten times fewer iterations than plain descent; AdaGrad and RMSProp stall at a constant learning rate (degree 6, $\kappa \approx 2000$).

4.7 Stochastic gradient descent (part h)

On the well-conditioned degree-2 problem (Figure 11), full-batch gradient descent converges geometrically to machine precision. Constant-rate SGD does not converge; it plateaus at a height that scales with the learning rate, because the iterate is a random walk in a noise cloud. A decaying schedule shrinks that cloud and keeps descending. A single-point step can even diverge in the ill-conditioned case, since it sees that one point's Hessian, which is why Adam with minibatches is the safe default.

4.8 Final comparison (part i)

Selecting each method by 5-fold cross-validation on Runge gives OLS at degree 12 (CV MSE ≈ 0.019), Ridge at degree 12 with a tiny penalty (≈ 0.021) and Lasso at degree 6 (≈ 0.027). OLS and Ridge are essentially tied and Lasso is slightly higher; all three follow Runge's peak (Figure 12). Read through (6), OLS controls only the degree, Ridge attacks the variance by shrinking the small-singular-value modes, and Lasso attacks the variance and also cuts the effective complexity by zeroing coefficients. On this smooth, low-dimensional problem with $n \gg p$ the penalties buy little over a cross-validated OLS degree; the sparsity of Lasso would pay off with many irrelevant features.

The degree of the minimum should not be read too literally. The cross-validation curve is flat over a range of degrees, and the bootstrap of part c and the cross-validation of part d place the minimum

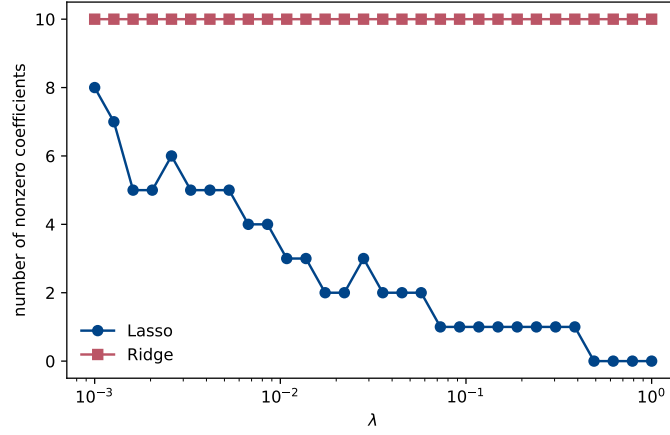


Figure 10: Lasso drives coefficients to zero one by one as λ grows, ending with a sparse model, while Ridge keeps all ten and only shrinks them (degree 10).

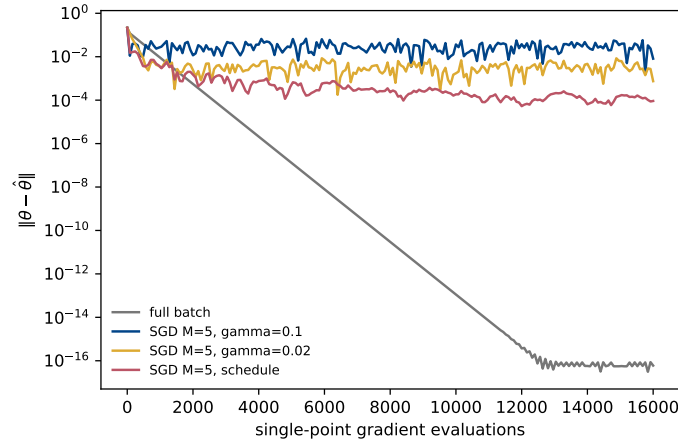


Figure 11: Full-batch gradient descent converges geometrically; constant-rate SGD plateaus at a height set by the learning rate, and a decaying schedule keeps descending (degree 2).

at different degrees while estimating the same expected test error. The reproducible result is the MSE level, about 0.02 with a spread of a few thousandths, not the exact degree: a different seed moves the degree of the minimum but leaves the level and the mechanism unchanged. Within the flat region the one-standard-error rule (Section 3.14) selects the simplest model whose cross-validated error is within one standard error of the minimum. For OLS with $k = 5$ the minimum is at degree 12 (CV MSE 0.019, standard error 0.002), and the rule selects degree 10 (CV MSE 0.021) instead: the two are statistically indistinguishable, and the simpler model is preferred. The seed 2026 is stated throughout so every figure can be regenerated.

Table 2: Cross-validated model selection on Runge ($k = 5$, seed 2026). The last column names the module and function that produced each fit.

Method	Best model	CV MSE	Code
OLS	degree 12	0.019	OLS.py: <code>ols_svd</code>
Ridge	degree 12, $\lambda \approx 10^{-5}$	0.021	Ridge.py: <code>ridge_closed_form</code>
Lasso	degree 6, $\lambda \approx 8 \times 10^{-4}$	0.027	LASSO.py: <code>lasso_ista</code>

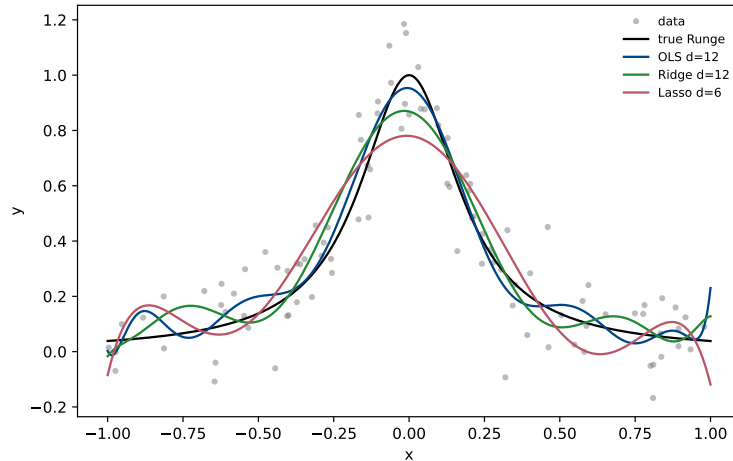


Figure 12: All three cross-validated models follow Runge’s peak; OLS and Ridge are essentially tied and Lasso is slightly smoother at the edges, refitted on all data against the true function and the noisy sample.

5 Conclusion and perspectives

We built and validated our own OLS, Ridge and Lasso solvers and a family of gradient-descent optimisers, and applied them to Runge’s function. The bootstrap made the bias–variance tradeoff explicit: the variance rises steeply with the degree while the bias stays low, so the test error is a U and its minimum is the balance point. Cross-validation located that minimum without spending a test set, and the three methods reached almost the same cross-validated error, OLS and Ridge essentially tied and Lasso a little higher. Ridge extends the usable degree because it damps the small-singular-value directions where the OLS coefficients blow up; Lasso sets coefficients to zero because the ℓ_1 penalty has a corner there; and gradient descent needs standardised features because the stability bound and the condition number are set by the eigenvalues of the Hessian, which the column scales distort.

The results are least certain in three places, and we state them so they are not left for the reader to find. The bias–variance estimate of part c measures the bias against the noisy test targets rather than the unknown f , so the reported bias absorbs the noise variance and is an upper estimate. The optimiser comparison of part f was read at a fixed iteration count rather than to a tolerance, and the learning rate was scanned on a grid of only sixteen values, coarse enough that we do not claim the exact ranking of the methods near their best γ . And parts a and b rest on a single train/test split, whose test curve is visibly jagged; the resampling of parts c, d and i is the trustworthy part of the model selection, and the single-split numbers should be read as illustrative.

Two next steps follow concretely. A validation-based stopping rule for SGD would remove the iteration count as a hyperparameter and make the optimiser comparison fairer. And the same pipeline applied to a higher-dimensional problem with many irrelevant features would let Lasso’s sparsity pay off, which it cannot on this smooth one-dimensional function; that is also the bridge to Project 2, where the linear model is replaced by a neural network, convexity is lost, and the gradient-descent machinery built here becomes essential.

What we take from the project is three things we did not see as clearly before the numerical work. First, the condition number is the quantitative bridge between high polynomial degree and the blow-up of the OLS coefficients, so the instability that motivates Ridge and Lasso is already visible in part a rather than appearing only once the penalties are introduced. Second, the exact degree of the cross-validation minimum is a noisy quantity, while its level is reproducible across seeds; reporting the level and the one-standard-error degree is the honest reading. Third, on a smooth low-dimensional target the regularised methods add little over a cross-validated OLS degree, which sharpened our sense of when shrinkage and sparsity are actually worth using.

A Use of AI/LLM tools

Large language models are permitted and encouraged in this course; using one does not reduce the author’s responsibility for the submission, and every result, derivation, code block and claim here is something the author understands, has verified and owns. This appendix declares the use in the format of the course guidelines. The contribution levels below are the author’s honest assessment and should be read as such.

A.1 Tools used. Claude (Opus 4.8, claude.ai, September–October 2026), used for the work described below; and GitHub Copilot (in VS Code, during the weekly exercises of weeks 34–38 whose solutions the modules are ported from). All prompts and direction from the author.

A.2 Text writing and editing. Levels: 0 none, 1 language only, 2 editorial, 3 generative (the model drafted substantial text from the author’s prompts; the author revised, verified and owns it).

Section	Level	Notes
Abstract	3	Drafted by the model from the author’s results; revised and verified by the author.
Introduction	3	Drafted from the author’s outline; the author checked the claims and citations.
Theory / Methods	3	Derivations drafted by the model from the week-36 exercises; the author checked every step against the lecture notes.
Code and validation	2	Rewritten for clarity by the model; content supplied by the author.
Results and discussion	3	Drafted from the author’s figures and numbers; the author verified every number against the notebooks.
Conclusion	3	Drafted from the author’s findings and chosen limitations; revised by the author.

Note on the Level-3 sections. These sections were drafted by Claude from the author’s own results, figures and outline, then revised, verified and taken over by the author. Representative prompts: for Theory, to write the bias–variance derivation following the week-36 exercises in the notation of this report, every step then checked against the lecture notes; for Results and discussion, to turn the figures and the numbers from the notebooks into the per-part discussion, every number then verified against the notebook output. The author changed wording, corrected emphasis and chose the limitations stated in the conclusion.

A.3 Code generation and assistance. Levels: 0 none, 1 debugging, 2 snippet, 3 skeleton, 4 substantial (the model wrote most of the file; the author adapted, tested, commented and verified it). The modules are ports to the Runge dataset of the author’s own weekly-exercise solutions, which were themselves written with GitHub Copilot as an inline assistant; the Project 1 ports and the analysis were then produced with Claude. Both tools are attributed below, and each module names its lecture source in a comment.

File / notebook	Tool, level	Description
Weekly-exercise code (basis)	Copilot, 2	Inline suggestions and short blocks while the author wrote the week 34–38 solutions; integrated and tested by the author.
code/*.py modules	Claude, 4	Structured and written by the model from the author’s weekly solutions; the author tested each against the closed-form and library benchmarks.
code/notebooks/*.ipynb	Claude, 4	Analysis and figures generated by the model from the author’s design; the author ran, checked and owns the results.

Because the assistance was substantial and pervasive rather than confined to separable blocks, the in-code declaration is made at file level, as the guidelines allow for heavy, hard-to-separate use: every module carries an `LLM-assisted` note and its lecture source at the top, and every notebook carries an LLM note in its first cell, each consistent with this appendix.

A.4 What is not declared. Using the model as a tutor and then writing a passage in one’s own words, and a literature search whose citations were verified against the sources, need not be declared; the references in this report were checked against the sources, not against the model.

B Numerical setup

The choices below are collected here so the main text stays readable and the runs can be reproduced. All randomness uses the seed 2026.

Global. Runge $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$; $n = 100$ points, x drawn uniformly, Gaussian noise $\sigma = 0.1$; features standardised on the training statistics, target centred, no intercept column; 80/20 train/test split where a single split is used; scores MSE and R^2 .

Part	Numerical choices
a, OLS	degrees 1–15; own SVD solver (pinv as cross-check).
b, Ridge	$1/n$ convention, penalty $n\lambda I$ (scikit-learn $\alpha = n\lambda$); $\lambda \in \{0, 10^{-4}, 10^{-2}, 1\}$ for the degree sweep and λ on log-grid $[10^{-6}, 10^2]$ at degree 12.
c, bootstrap	100 resamples; degrees 1–14; $n \in \{40, 100, 400\}$; OLS only; scaling fixed on the training set before resampling.
d, CV	$k = 5$ and 10; own k -fold and scikit-learn <code>KFold</code> ; scaling inside folds; Ridge α on log-grid $[10^{-4}, 10^3]$ at degree 12.
e, GD	degree 4; $\gamma \in \{0.02, \gamma^*, 0.99\gamma_{\max}, 1.02\gamma_{\max}\}$; $\theta_0 = 0$; analytical gradient and <code>jax.grad</code> (x64).
f, optimisers	degree 6; $\gamma = 0.9\gamma_{\max}$ (plain, momentum), $\gamma = 0.1$ (adaptive); $\beta = 0.9$, $\rho = 0.99$, Adam $(\beta_1, \beta_2) = (0.9, 0.999)$, $\epsilon = 10^{-8}$; tolerance 10^{-6} , budget 2×10^4 ; 16-point γ scan.
g, Lasso	degree 10; ISTA with step $\gamma = 1/L$ ($L = \lambda_{\max}$ of the Hessian); λ on log-grid $[10^{-3}, 1]$; nonzero threshold 10^{-8} ; scikit-learn comparison with $\alpha = \lambda/2$.
h, SGD	degree 2; $M \in \{1, 5, 20, 100\}$; 200 epochs; schedule $(t_0, t_1) = (5, 50)$, $\gamma_0 = 0.1$.
i, selection	5-fold CV; OLS degrees 1–12; Ridge degree $\times \lambda$ (log-grid $[10^{-5}, 1]$, 12 values); Lasso degrees $\{4, 6, 8, 10, 12\} \times \lambda$ (log-grid $[10^{-4}, 1]$, 10 values); one-standard-error rule, $SE = \text{fold std}/\sqrt{k}$.

References

- [1] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018. <https://jmlr.org/papers/v18/17-468.html>.
- [2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Neca, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax>, 2018.

- [3] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011. <https://jmlr.org/papers/v12/duchi11a.html>.
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. <https://www.deeplearningbook.org>.
- [5] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. <https://doi.org/10.1038/s41586-020-2649-2>.
- [6] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2 edition, 2009. <https://doi.org/10.1007/978-0-387-84858-7>.
- [7] Morten Hjorth-Jensen. Applied data analysis and machine learning: Lecture notes. <https://github.com/CompPhysics/MachineLearning>, 2026.
- [8] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12:55–67, 1970. <https://doi.org/10.1080/00401706.1970.10488634>.
- [9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980, <https://arxiv.org/abs/1412.6980>.
- [10] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [11] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58:267–288, 1996. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.